

**LAPORAN PRAKTIKUM**  
**PEMROGRAMAN BERORIENTASI OBJEK**  
**MODUL 3 – KONSEP DASAR PEMROGRAMAN BERORIENTASI**  
**OBJEK**



Khairun Fahmi

103132400035

S1SD-05-A

**PROGRAM STUDI SAINS DATA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**  
**2026**

## A. Dasar Teori

Pemrograman Berorientasi Objek (OOP) adalah konsep pemrograman yang menggunakan class dan object untuk mengatur data dan fungsi dalam program. Class merupakan rancangan untuk membuat object, sedangkan object adalah hasil dari class.

Dalam OOP terdapat field untuk menyimpan data dan method untuk menjalankan proses. Constructor digunakan saat membuat object dan dapat digunakan untuk memberikan nilai awal pada object.

Enkapsulasi digunakan untuk membatasi akses terhadap data dalam class, salah satunya dengan modifier `private`. Selain itu, keyword `this` digunakan untuk merujuk pada object yang sedang digunakan, terutama ketika nama parameter sama dengan nama field.

## B. Guided

### 1. Method

#### PassedByValue

```
class PassedByValue {  
    public static void main(String[] args) {  
        int x, y;  
        TestPass z;  
  
        z = new TestPass(50, 100);  
        x = 10;  
        y = 20;  
  
        System.out.println("Nilai sebelum passed by value : ");  
        System.out.println("x = " + x);  
        System.out.println("y = " + y);  
  
        // passed by value  
        z.calculate(x, y);  
  
        System.out.println("Nilai sesudah passed by value : ");  
        System.out.println("x = " + x);  
        System.out.println("y = " + y);  
  
        System.out.println("Nilai sebelum passed by reference : ");  
        System.out.println("z.i = " + z.i);  
        System.out.println("z.j = " + z.j);  
  
        // passed by reference  
        z.calculate(z);  
  
        System.out.println("Nilai sesudah passed by reference : ");  
        System.out.println("z.i = " + z.i);  
        System.out.println("z.j = " + z.j);  
    }  
}
```

```
}  
}
```

## TestPass

```
class TestPass {  
    int i, j;  
  
    TestPass(int a, int b) {  
        i = a;  
        j = b;  
    }  
  
    // passed by value dengan parameter berupa tipe data primitif  
    void calculate(int m, int n) {  
        m = m * 10;  
        n = n / 2;  
    }  
  
    // passed by reference dengan parameter berupa tipe data class  
    void calculate(TestPass e) {  
        e.i = e.i * 10;  
        e.j = e.j / 2;  
    }  
}
```

## Output :

```
-----  
Nilai sebelum passed by value :  
x = 10  
y = 20  
Nilai sesudah passed by value :  
x = 10  
y = 20  
Nilai sebelum passed by reference :  
z.i = 50  
z.j = 100  
Nilai sesudah passed by reference :  
z.i = 500  
z.j = 50  
-----  
BUILD SUCCESS  
-----
```

## 2. Keyword “this”

### DemoLagu

```
package com.mycompany.praktikummodul3;  
  
public class DemoLagu {
```

```

        public static void main(String[] args) {
            Lagu a = new Lagu();

            a.IsiParam("God Will Make A Way", "Don Moen ");
            a.cetakKeLayar();
        }
    }

```

### Lagu

```

package com.mycompany.praktikummodul3;

public class Lagu {
    private String pencipta;
    private String judul;

    public void IsiParam(String judul, String pencipta) {
        this.judul = judul;
        this.pencipta = pencipta;
    }

    public void cetakKeLayar() {
        if (judul == null && pencipta == null)
            return;

        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}

```

### Output :

```

] --- exec:3.6.3:exec (default-cli) @ PraktikumModul3 ---
- Judul : God Will Make A Way, pencipta : Don Moen
-----
BUILD SUCCESS
-----
Total time:  2.122 s

```

### Buku

```

package com.mycompany.praktikummodul3;

public class Buku {
    private String pengarang;
    private String judul;

    private Buku() {
        this("Rumah Kita", "GoodBles");
    }
}

```

```

private Buku(String judul, String pengarang) {
    this.judul = judul;
    this.pengarang = pengarang;
}

private void cetakKeLayar() {
    System.out.println("Judul : " + judul + " Pengarang : " + pengarang);
}

public static void main(String[] args) {
    Buku a, b;

    a = new Buku("Jurassic Park", "Michael Chricton");
    b = new Buku();

    a.cetakKeLayar();
    b.cetakKeLayar();
}
}

```

Output:

```

Judul : Jurassic Park Pengarang : Michael Chricton
- Judul : Rumah Kita Pengarang : GoodBles
-----
BUILD SUCCESS
-----
Total time:  1.753 s

```

### C. Unguided

#### 1. Soal Unguided

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Jawaban :

### UnguidedModul3

```
package com.mycompany.praktikummodul3;

import java.util.Arrays;

public class UnguidedModul3 {
    public static void main(String[] args) {
        double[] suhuHarian = {
            30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9
        };

        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();

        int indexKosong = pengolah.cariIndexKosong();

        System.out.println();
        System.out.println("Index hari kosong (dimulai dari 0): " + indexKosong);

        pengolah.isiDataKosong();

        System.out.println();
        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();

        System.out.printf("%nRata-rata : %.2f°C%n", pengolah.hitungRataRata());

        System.out.println();
        System.out.println("Isi array suhuHarian di main setelah isiDataKosong() dijalankan:");
        System.out.println(Arrays.toString(suhuHarian));

        /*
         * Array suhuHarian di main ikut berubah karena array merupakan
         * tipe data reference. Constructor PengolahSuhu menyimpan
         * referensi ke array yang sama, bukan membuat salinan array.
         * Jadi ketika isiDataKosong() mengubah isi array melalui object
         * PengolahSuhu, perubahan tersebut juga terlihat pada array
         * suhuHarian yang berada di main.
         */
    }
}
```

### PengolahSuhu

```
package com.mycompany.praktikummodul3;

public class PengolahSuhu {
```

```

private double[] suhuHarian;
private static final double NILAI_KOSONG = -1.0;

public PengolahSuhu(double[] suhuHarian) {
    this.suhuHarian = suhuHarian;
}

public void tampilkanData() {
    for (int i = 0; i < suhuHarian.length; i++) {
        if (suhuHarian[i] == NILAI_KOSONG) {
            System.out.println("Hari " + (i + 1) + " : (kosong)");
        } else {
            System.out.printf("Hari %d : %.1f°C%n", i + 1, suhuHarian[i]);
        }
    }
}

public int cariIndexKosong() {
    for (int i = 0; i < suhuHarian.length; i++) {
        if (suhuHarian[i] == NILAI_KOSONG) {
            return i;
        }
    }

    return -1;
}

public void isiDataKosong() {
    int index = cariIndexKosong();

    if (index != -1) {
        suhuHarian[index] =
            (suhuHarian[index - 1] + suhuHarian[index + 1]) / 2;
    }
}

public double hitungRataRata() {
    double total = 0;

    for (double suhu : suhuHarian) {
        total += suhu;
    }

    return total / suhuHarian.length;
}

```

Output :

```
--- exec:3.6.3:exec (default-cli) @ PraktikumModul3 ---
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
-----
BUILD SUCCESS
-----
```

Penjelasan :

Program tersebut digunakan untuk mengolah data suhu harian selama 7 hari. Data suhu disimpan dalam array suhuHarian, sedangkan nilai -1.0 digunakan sebagai penanda bahwa terdapat data suhu yang kosong. Class PengolahSuhu memiliki beberapa method untuk menampilkan data, mencari posisi data yang kosong, mengisi data kosong berdasarkan rata-rata suhu sebelum dan sesudahnya, serta menghitung rata-rata suhu setelah data diperbaiki. Nilai NILAI\_KOSONG dibuat sebagai static final agar nilai penanda hanya ditulis satu kali dan tidak dapat diubah. Array dari main diberikan ke object melalui constructor, sehingga object PengolahSuhu menggunakan array yang sama. Oleh karena itu, ketika data kosong diisi melalui method isiDataKosong(), isi array yang ada di main juga ikut berubah.

#### D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep dasar Pemrograman Berorientasi Objek pada Java dapat diterapkan menggunakan class, object, field, method, dan constructor. Praktikum juga menunjukkan penggunaan enkapsulasi dengan modifier private, penggunaan class field dan instance field, serta keyword this untuk mengakses field dan memanggil constructor lain. Selain itu, melalui percobaan pass by value dan array atau object dapat dipahami bahwa perubahan data bergantung pada cara data tersebut diberikan kepada method. Penerapan konsep-konsep tersebut dapat membuat program lebih terstruktur, terorganisir, dan mudah dipahami.